学习会话 · 产品设计说明

1. 这台工作站在做什么

StarMap 是一个**个人学习工作站**。它不按章节从头讲课，而是先弄清楚「你现在卡在哪」，再给一张能立刻上手的卡片。

学习者只需要说一句话，比如「明天考数电，布尔代数是必考」。系统接下来做的事是：

1. **问两三个问题**——确认学过没有、是为了考试还是为了真懂；
2. **定一条路线**——从第几段开始讲、讲多细、这一关几分钟；
3. **带着练**——边讲边出题，错了当场指出错在哪一步；
4. **给结论**——这一关过了没有、掌握度变成多少、下次什么时候回来。

学习者全程只做两件事：**说话**，或**点卡片上的选项**。没有菜单要背，没有章节要选。

2. 五条设计理念

这一章是整个产品的地基。每条都回答同一个问题：**为什么这样做，而不是那样做。**

2.1 关卡：把「一章」拆成能走完的七段

传统教材一章三十页，学习者不知道自己走到哪、还剩多少。我们把一个知识点切成**七个阶段**：

```
引导（为什么学它） → 展示细节（一个完整例子） → 简单问题（先确认看懂了）
→ 深究（换个问法，逼自己想） → 进一步讲解（补上刚才卡的那一步）
→ 判定（这一关算不算过） → 小结（真正立起来的是哪几件事）
```

每一段只做一件事，做完立刻知道下一段是什么。**关卡不是「练习题合集」，而是一条有节奏的路**：讲解和练习交替出现，学习者不会连看十分钟视频，也不会连做十道题才发现自己方向错了。

关卡还有一条硬规则：**判定段单独算数。**讲解段随便看、随便翻，只有判定段的结果进入掌握度——这样「看完」和「学会」才分得开。

2.2 借鉴百万 star 项目的追问设计，摸清真实起点

「你觉得你什么水平」——这个问题拿不到任何信息。学习者的答案永远是「还行吧」，这是一个防御性回答。

所以我们的追问方式，借鉴了 GitHub 上**百万 star 技能库**里一套被大量项目验证过的设计思路：不问抽象立场，问**最近一次具体遭遇。**

不这样问	这样问
你的物理基础怎么样？	你手边现在能拿到的是什么——能自己导出的谱数据、一张别人给的截图、能碰真机，还是只有书和讲义？
你想学到什么程度？	最近是哪一次具体的遭遇让你想起「能量分辨率」这个词的？

区别在于：前者让人编一个答案，后者让人**回忆一件真事。**回忆出来的东西骗不了人，而编出来的答案我们也没法用。

2.3 先问「你为什么学」，再决定怎么教

同一个知识点，为了考试和为了懂，教法完全不同：

- **为了考试**：直接进练习，错的地方指到具体一项，不讲原理推导；
- **为了真懂**：先给完整例子，再让他自己补最后一步；
- **为了用**：从「你手里那台设备」出发，先讲清参数怎么读。

所以系统在开口后的前几轮里，一定会问出**真实意图**，然后把它写进这一关的判定依据。意图不清就不往下走——**教学目标没立起来之前，讲公式是浪费双方的时间。**

2.4 苏格拉底式提问：不代做，只往前推一步

学习者说「我不会」时，最容易犯的错是直接给答案。我们的做法是：

1. **先分清是不会，还是没看懂题**——这两件事的补救方式完全不同；
2. **一次只问一个问题**，不超过两句；
3. **追问不超过两层**，两层之后还没进展，就降一层问更基础的；
4. 学习者说「直接告诉我」时，**给一个小结论，但要说清为什么这一步可以省。**

这不是为了为难人。真正的目的是：让学习者**自己跨过那一步**——跨过去之后，那一步就变成他的了；被直接告知的，三天后会原样还回来。

2.5 进度实时同步：让人随时知道「我在哪」

学习最伤人的不是难，是**不知道自己有没有在进步。**

所以右栏的每个数字都是实时的、可查的：

看到什么	意味着什么
掌握度 **66% → 84%**	这一关的作答证据把它推高了 18 个点
作答 8/8 题 • 覆盖 3/3 组 • 连对 2/2	三条门槛各自到哪了，不是最后才弹一个分数
七段走到哪了	当前在第几段，后面还有几段
本次对话事实	这一轮检索了几段教材、放行了哪几段、调用了哪几个技能
下次复验：1 天后	什么时候该回来，不用自己记

进度条不是装饰，它是**信任的载体**：数字对不上，人就再也不信这个系统了。所以每一个数字都指得出来源——出自哪本书、第几页、哪一次作答。

2.6 错题分析：AI 深度结合，而不是只判对错

答错之后只说「答错了」，等于什么也没做。系统对每道错题做三件事：

1. **判断错因类型**——是概念混了（把「或」当成「与」）、计算失误，还是压根没读懂题；
2. **只问一个关键点**，让学习者自己把这一步说出来，而不是再讲一遍；
3. **写进复验队列**——错过的题一定会再来找你，不因为通关就消失。

错因判断是 AI 在这一层真正吃重的地方：**同一个错误答案，背后可能是完全不同的两种缺陷**，补救方式也完全不同。分不清这一点，「智能批改」就只是把红叉换成了红字。

---

3. 三种人，同一套流程

谁	他会怎么说	系统怎么应对
完全没学过	「我完全没学过，从哪开始」	先给完整例子走一遍，再自己动手；一轮只推 2 个新知识点
学过但卡住	「我会一点，但一化简就乱」	不讲整章，只把卡住那一步挑出来问；一轮推 3 个
考前突击	「明天就考，只想练最容易错的」	跳过讲解直接进练习，错处指到具体一项；一轮推 4 个

三者进的是**同一个知识点**，用的是**同一套判据**——差别只在起点和节奏。系统不给人贴标签，只调整「从哪儿开始、讲多细」。

4. 一轮对话怎么进行

这是最核心的循环，一次十几秒：

```
学习者说一句话
   ↓
系统先「想」—把这一轮要确认的事写清楚（约 2 秒，边想边显示思路）
   ↓
系统回答，并给出下一张卡片
   ↓
学习者二选一：点卡片上的选项，或直接在输入框说一句
   ↓
系统先确认「我记下了」，再进入下一轮
```

三个刻意的设计：

- **先想再答**：思考必须在回答**之前**出现。顺序反了，用户会认为答案是早就编好的。
- **两条回答通道**：有人愿意点，有人想解释自己的思路。两条路都推进，并且都会回一句「记下了：你选的是… / 你写的是…」——让人确认自己被听见。
- **一次只问一件**：卡片上永远只有一个问题、一张卡。学习者的注意力是有限的，一次给两张卡，等于一张都没给。

5. 功能清单

5.1 学会一件事的完整链路

功能	解决什么	学习者看到什么
开口即开始	不用先选章节	说一句话，系统就开始问
基线确认	避免无效自评	两三个具体问题，问的是「最近一次遭遇」
路线与节奏	不同人不用同一条路	一张关卡卡：几分钟、几道题、从哪一段开始
七段教学	讲解与练习交替	引导 → 例子 → 简单题 → 深究 → 再讲 → 判定 → 小结
错题修正	错了不能只对答案	先判错因，再只问一个关键点，让学习者自己说出来
通关结论	让人知道到底过没过	三条标准全部满足才算过，附掌握度变化与下次复习时间

5.2 九个页面，一条主线

学习页是入口，其余八页是「查证」用的：

页面	用来干什么
学习	对话与闯关（主战场）
定级	用几道题测出当前档位
复验	到期该复习的知识点排队
看板	今天学什么、为什么是它
知识地图	看一个知识点前后左右连着谁
关卡市场	挑一关练，或把自己学的存成关卡
数据库	每个知识点出自哪本书、第几页
内容库	库里有哪些教材、题库、技能
技能库	系统会哪几种教法，各在什么场景用

6. 算不算学会

不设「答对 80% 就过关」这种粗暴阈值，而是三条同时满足：

门槛	防的是什么
至少 8 道题	防运气好——题太少，对错都说明不了什么
覆盖 3 个变式	防只会背这一道——同一知识点换三种问法
连续答对 2 次	防蒙对一次

错题不进判定池，但会进复验队列——**错过的题一定会再来找你。**复验按 1 / 3 / 7 / 21 / 60 天阶梯推进：答对往上走一层，答错往回退一层，但**历史最高掌握度永不下降。**

这条规则的目的不是给分，是**让人敢答错。**

7. 效果

7.1 一次典型的考前突击

阶段	发生了什么
开口	「我想学能量分辨率，实验室那台 HPGe 的峰看着比别处胖」
基线	系统问两个问题：手边能拿到什么数据？更想先搞公式还是先看谱？
定线	给定一条：从「峰形为什么变宽」入手，4 题、14 分钟
教学	七段推进；两处错题被当场诊断（把「+」当成「·」、把 FWHM 变宽当成分辨率变差）
通关	掌握度 66% → 84%，三条门槛全满足，1 天后自动安排复验

7.2 可观测的结果

- **一轮十几秒**，不用等几十秒才看到反馈；
- **通关判定可解释**：三条门槛的进度实时可见，不是最后弹一个分数；
- **九页随时可切**，回来对话接着走，不丢进度；
- **每个数字都能查**：掌握度、题数、变式数、连对数，都指得出是哪条证据。

8. 产品规则（几条不妥协的）

1. **档位不能自选。**档位是测量结果，想升级只能多做题。
2. **错题要回来。**错过的题一定进复验队列，不因为通关就消失。
3. **历史最高不下降。**答错只影响当前熟练度。
4. **禁用的技能连名字都不出现。**不靠「告诉模型别用」来约束行为，而是根本不把那条指令发出去。
5. **讲解段不进判定池。**看完不算学会，只有判定段的结果算数。